ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

ASSIGNMENT – 05

NAME – SUFIYA AFREEN

ENROLLMENT NO – 2403A51067

BATCH – CSE (03)

YEAR/SEM – 3/1

DATE – 19-8-26

Lab/Exp 5: Applying Optimizers -Adam, SGD and Observing Convergence.

Kaggle Dataset Link: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

Tasks:

1. Load and preprocess the MNIST dataset/example.
2. Build the required PyTorch model/program.
3. Train/execute the model.
4. Evaluate using suitable metrics.
5. Visualize results using plots where applicable.
6. Write a brief conclusion comparing observations.

CODE –

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

import matplotlib.pyplot as plt
import numpy as np

# For reproducibility
torch.manual_seed(42)

# Check device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# Convert images to tensors and normalize them
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Training dataset
train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

# Testing dataset
test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    transform=transform,
    download=True
)

# Data loaders
batch_size = 128

train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False
)

print("Training samples:", len(train_dataset))
print("Testing samples :", len(test_dataset))
```

```python
fig, axes = plt.subplots(2, 5, figsize=(10, 5))

for i, ax in enumerate(axes.flat):
    image, label = train_dataset[i]

    # Unnormalize for displaying
    image = image * 0.3081 + 0.1307

    ax.imshow(image.squeeze(), cmap="gray")
    ax.set_title(f"Label: {label}")
    ax.axis("off")

plt.tight_layout()
plt.show()

class MNIST_Net(nn.Module):

    def __init__(self):
        super(MNIST_Net, self).__init__()

        self.network = nn.Sequential(

            # Input: 28 x 28 = 784
            nn.Flatten(),

            # Hidden Layer 1
            nn.Linear(28 * 28, 128),
            nn.ReLU(),

            # Hidden Layer 2
            nn.Linear(128, 64),
            nn.ReLU(),

            # Output Layer: 10 classes (0-9)
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.network(x)


def train_model(optimizer_name, epochs=5):

    # Create a fresh model
    model = MNIST_Net().to(device)

    # Loss function
    criterion = nn.CrossEntropyLoss()

    # Select optimizer
    if optimizer_name == "SGD":

        optimizer = optim.SGD(
            model.parameters(),
            lr=0.01
        )
```

```
    else:
        raise ValueError("Unknown optimizer")

    train_losses = []
    train_accuracies = []

    print("\n==================================")
    print("Training using", optimizer_name)
    print("==================================")

    for epoch in range(epochs):

        model.train()

        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:

            images = images.to(device)
            labels = labels.to(device)

            # Clear old gradients
            optimizer.zero_grad()

            # Forward pass
            outputs = model(images)

            # Calculate loss
            loss = criterion(outputs, labels)

            # Backpropagation
            loss.backward()

            # Update weights
            optimizer.step()

            # Statistics
            running_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        epoch_loss = running_loss / len(train_loader)
        epoch_accuracy = 100 * correct / total

        train_losses.append(epoch_loss)
        train_accuracies.append(epoch_accuracy)

        print(
            f"Epoch [{epoch+1}/{epochs}] "
            f"Loss: {epoch_loss:.4f} "
            f"Accuracy: {epoch_accuracy:.2f}%"
        )
```

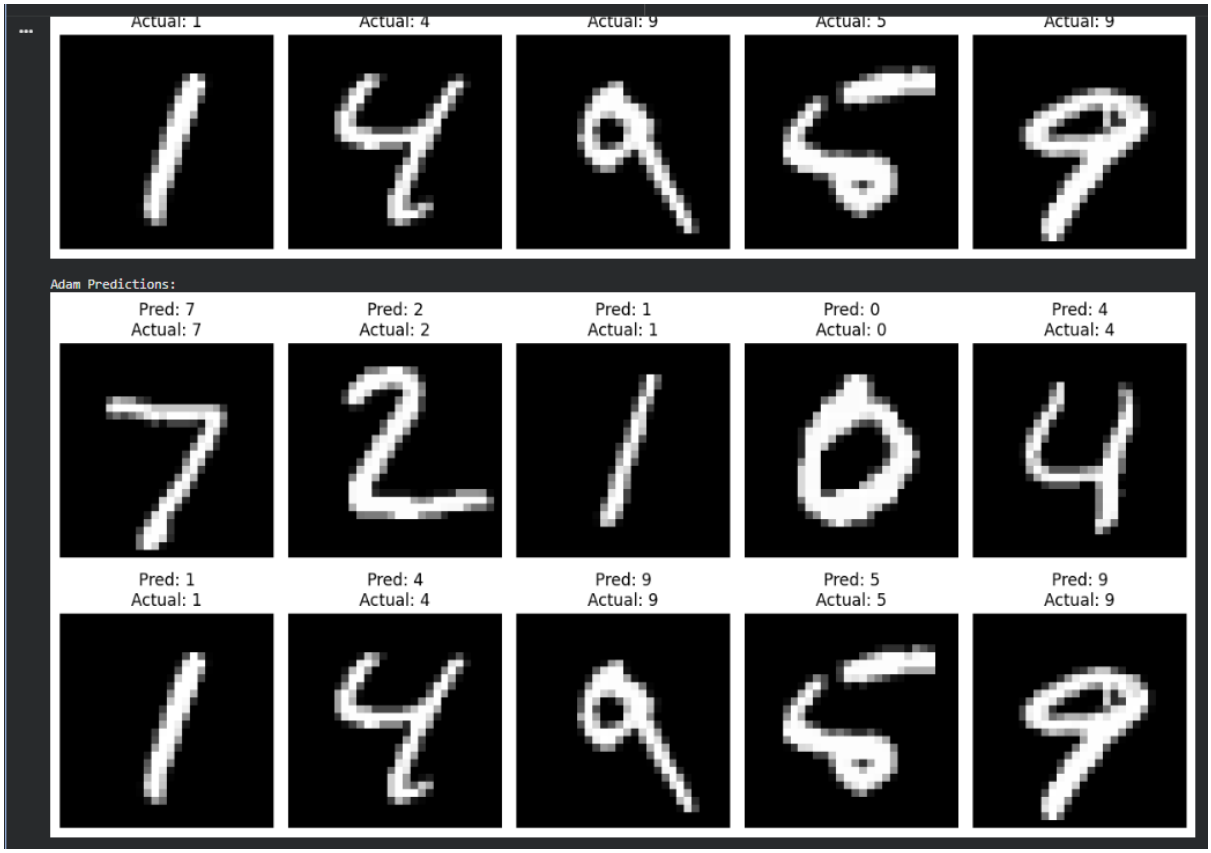
```
    return model, train_losses, train_accuracies

sgd_model, sgd_losses, sgd_accuracies = train_model(
    "SGD",
    epochs=5
)



adam_model, adam_losses, adam_accuracies = train_model(
    "Adam",
    epochs=5
)


def evaluate_model(model):

    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():

        for images, labels in test_loader:

            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)

            _, predicted = torch.max(outputs.data, 1)

            total += labels.size(0)

            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total

    return accuracy

sgd_test_accuracy = evaluate_model(sgd_model)
adam_test_accuracy = evaluate_model(adam_model)

print("\n==================================")
print("FINAL TEST RESULTS")
print("==================================")

print(f"SGD  Test Accuracy : {sgd_test_accuracy:.2f}%")
print(f"Adam Test Accuracy : {adam_test_accuracy:.2f}%")


epochs_range = range(1, 6)

plt.figure(figsize=(8, 5))
```

```
plt.plot(
    epochs_range,
    sgd_losses,
    marker="o",
    label="SGD"
)

plt.plot(
    epochs_range,
    adam_losses,
    marker="o",
    label="Adam"
)

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Convergence Comparison: SGD vs Adam")
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    epochs_range,
    sgd_accuracies,
    marker="o",
    label="SGD"
)

plt.plot(
    epochs_range,
    adam_accuracies,
    marker="o",
    label="Adam"
)

plt.xlabel("Epoch")
plt.ylabel("Training Accuracy (%)")
plt.title("Training Accuracy: SGD vs Adam")
plt.legend()
plt.grid()
plt.show()


optimizers = ["SGD", "Adam"]
accuracies = [sgd_test_accuracy, adam_test_accuracy]

plt.figure(figsize=(7, 5))

plt.bar(optimizers, accuracies)

plt.xlabel("Optimizer")
plt.ylabel("Test Accuracy (%)")
plt.title("Test Accuracy Comparison")
```

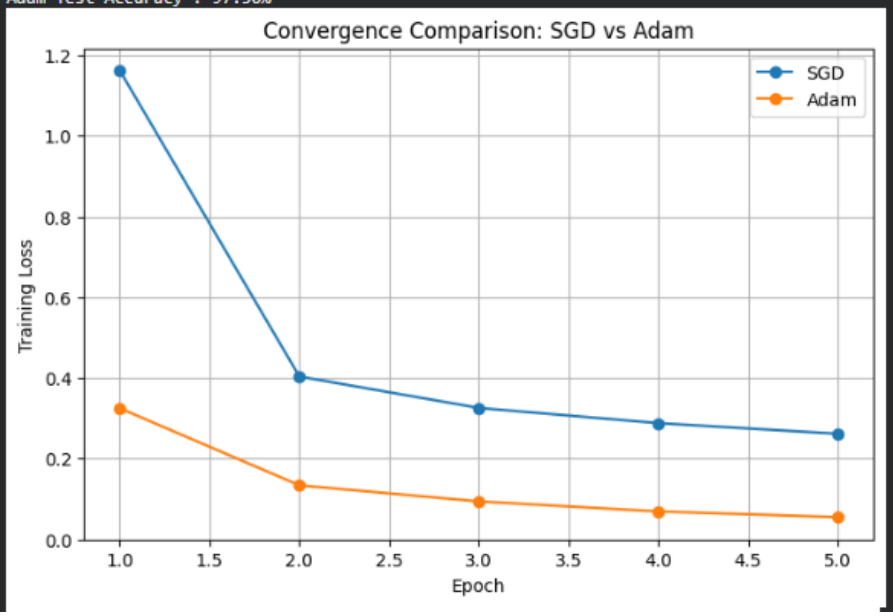
```python
for i, value in enumerate(accuracies):
    plt.text(
        i,
        value + 0.2,
        f"{value:.2f}%",
        ha="center"
    )

plt.ylim(0, 100)
plt.show()

def show_predictions(model):

    model.eval()

    images, labels = next(iter(test_loader))

    images = images.to(device)
    labels = labels.to(device)

    with torch.no_grad():
        outputs = model(images)
        _, predictions = torch.max(outputs, 1)

    images = images.cpu()
    predictions = predictions.cpu()
    labels = labels.cpu()

    plt.figure(figsize=(12, 6))

    for i in range(10):

        plt.subplot(2, 5, i + 1)

        image = images[i] * 0.3081 + 0.1307

        plt.imshow(image.squeeze(), cmap="gray")

        plt.title(
            f"Pred: {predictions[i].item()}\n"
            f"Actual: {labels[i].item()}"
        )

        plt.axis("off")

    plt.tight_layout()
    plt.show()


print("\nSGD Predictions:")
show_predictions(sgd_model)

print("\nAdam Predictions:")
show_predictions(adam_model)
```

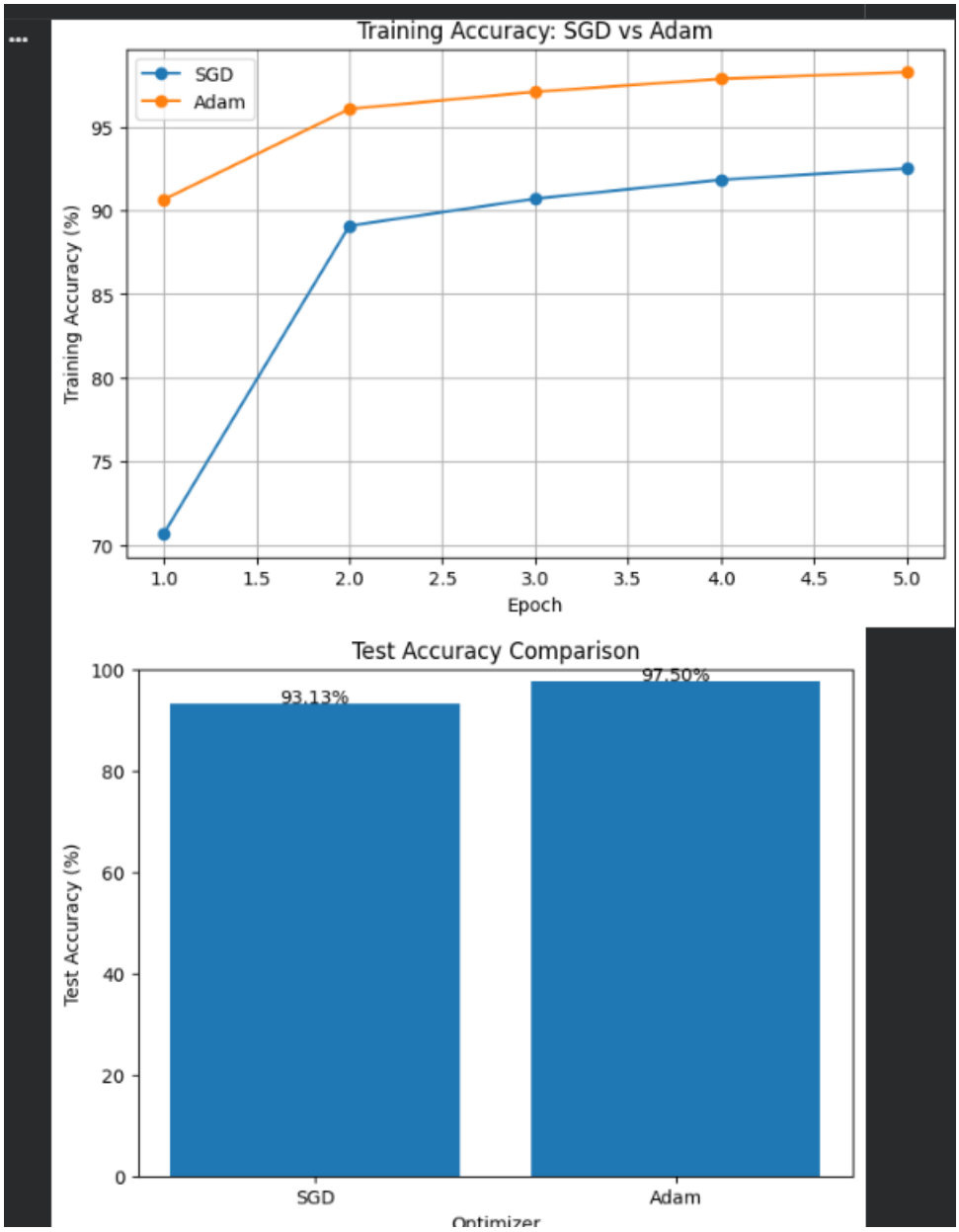
```
print("\n==================================")
print("CONCLUSION")
print("==================================")

if adam_test_accuracy > sgd_test_accuracy:
    print(
        "Adam achieved higher test accuracy than SGD in this experiment."
    )
else:
    print(
        "SGD achieved higher or equal test accuracy than Adam in this experiment."
    )

print(
    "Adam generally converges faster because it adapts the learning rate "
    "for individual parameters."
)

print(
    "SGD is simpler and can also achieve good performance with an appropriate "
    "learning rate."
)

print(
    "The loss and accuracy plots show the convergence behavior of both "
    "optimizers over the training epochs."
)
```

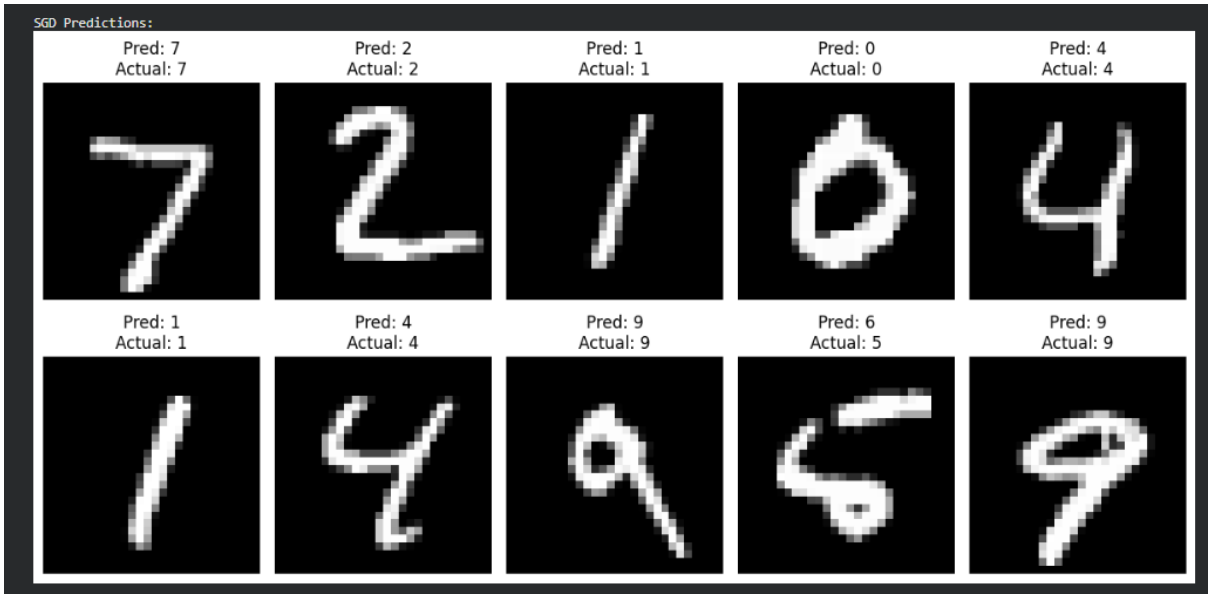
OUTPUT-

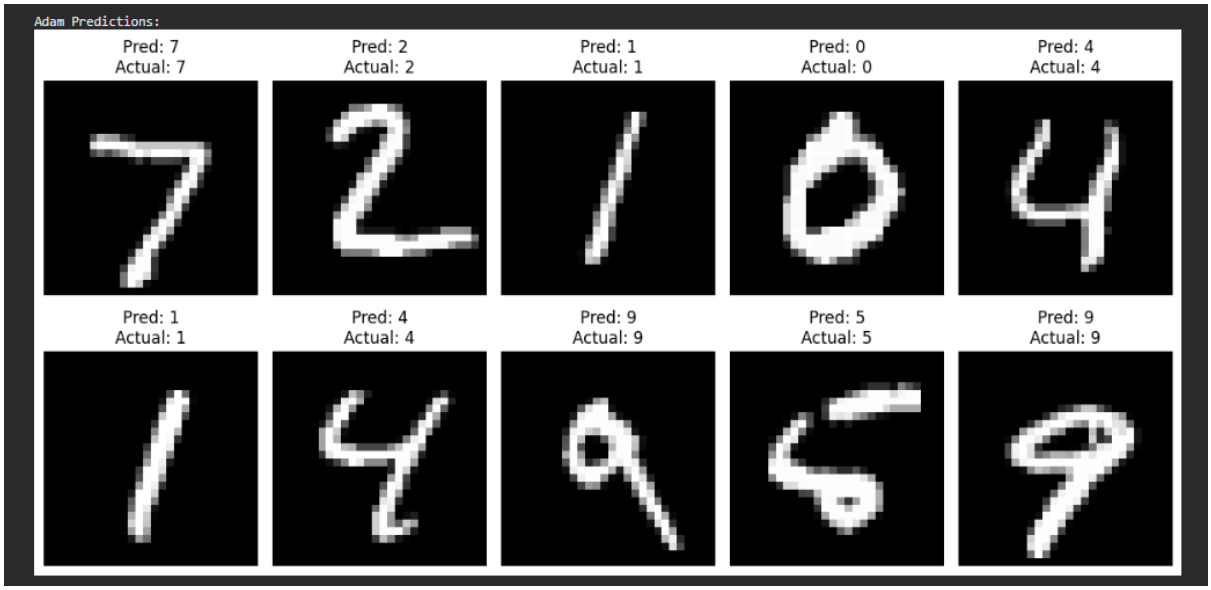


```
==================================
Training using SGD
==================================
Epoch [1/5] Loss: 1.1616 Accuracy: 70.66%
Epoch [2/5] Loss: 0.4035 Accuracy: 89.08%
Epoch [3/5] Loss: 0.3257 Accuracy: 90.70%
Epoch [4/5] Loss: 0.2880 Accuracy: 91.83%
Epoch [5/5] Loss: 0.2617 Accuracy: 92.51%

==================================
Training using Adam
==================================
Epoch [1/5] Loss: 0.3246 Accuracy: 90.64%
Epoch [2/5] Loss: 0.1338 Accuracy: 96.06%
Epoch [3/5] Loss: 0.0939 Accuracy: 97.09%
Epoch [4/5] Loss: 0.0691 Accuracy: 97.86%
Epoch [5/5] Loss: 0.0549 Accuracy: 98.27%

==================================
FINAL TEST RESULTS
==================================
SGD  Test Accuracy : 93.13%
Adam Test Accuracy : 97.50%
```




Training Accuracy: SGD vs Adam
SGD
Adam
Training Accuracy (%)
Epoch
95
90
85
80
75
70
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
Test Accuracy Comparison
93.13%
97.50%
Test Accuracy (%)
100
80
60
40
20
0
SGD
Adam
Optimizer





CONCLUSION –

```
==================================
CONCLUSION
==================================
Adam achieved higher test accuracy than SGD in this experiment.
Adam generally converges faster because it adapts the learning rate for individual parameters.
SGD is simpler and can also achieve good performance with an appropriate learning rate.
The loss and accuracy plots show the convergence behavior of both optimizers over the training epochs.
```